

# Instruções para Implementação do Sistema de Edição e Exclusão

## 1. Modificações no Sistema\_Entrada\_Dados.py

### 1.1 Adicionar as novas classes no arquivo

Adicione as seguintes classes **no final** do arquivo `Sistema_Entrada_Dados.py`, antes da execução principal:

python

```
# Cole aqui o código das classes:  
# - GerenciadorLancamentos  
# - EditorLancamentoCompleto
```

### 1.2 Modificar a classe SistemaEntradaDados

No método `__init__`, adicione após a linha onde inicializa outros componentes:

python

```
def __init__(self, parent=None):  
    # ... código existente ...  
  
    # Adicionar esta linha  
    self.gerenciador_lancamentos = None  
  
    # ... resto do código existente ...
```

**Adicionar os novos métodos na classe SistemaEntradaDados:**

python

```
def adicionar_botao_gerenciar_lancamentos(self):
    """Adiciona botão para gerenciar lançamentos na aba de fornecedor"""
    frame_gerenciamiento = ttk.LabelFrame(self.aba_fornecedor, text="Gerenciamento de Dados")
    frame_gerenciamiento.pack(fill='x', padx=10, pady=5)

    frame_botoes_ger = ttk.Frame(frame_gerenciamiento)
    frame_botoes_ger.pack(fill='x', padx=5, pady=8)

    ttk.Button(
        frame_botoes_ger,
        text="Gerenciar Lançamentos",
        command=self.abrir_gerenciar_lancamentos,
        style='Medium.TButton'
    ).pack(side='left', padx=5)

def abrir_gerenciar_lancamentos(self):
    """Abre o gerenciador de lançamentos"""
    if not hasattr(self, 'gerenciador_lancamentos') or self.gerenciar_lancamentos is None:
        self.gerenciar_lancamentos = GerenciadorLancamentos(self)
    self.gerenciar_lancamentos.abrir_gerenciar()
```

**No método** `setup_aba_fornecedor`, adicione **no final** do método, antes da linha que cria o frame de botões na parte inferior:

python

```
def setup_aba_fornecedor(self):
    # ... código existente ...

    # Adicionar antes do frame_botoes_fornecedor
    self.adicionar_botao_gerenciar_lancamentos()

    # Frame de botões gerais na parte inferior (código existente)
    frame_botoes_fornecedor = ttk.Frame(self.aba_fornecedor)
    # ... resto do código existente ...
```

**Modificar o método** `verificar_duplicidade_antes_salvar`:

Substitua o método existente pela versão modificada que considera o status:



```

def verificar_duplicidade_antes_salvar(self, sheet, dados):
    """
    Versão modificada que considera status de exclusão
    """
    logger = system_logger.get_logger()
    logger.debug(f"Verificando duplicidade para: {dados['nome']} - {dados['referencia']} - Valc

for row in sheet.iter_rows(min_row=2, values_only=True):
    # Pular Linhas vazias
    if not row[0]:
        continue

    # Verificar status - pular se excluído (coluna 14 = índice 13)
    status = row[13] if len(row) > 13 else 'ATIVO'
    if status == 'EXCLUIDO':
        continue

    # Resto do código permanece igual...
    try:
        valor_planilha = float(str(row[8]).replace(',', ' '))
        valor_novo = float(str(dados['valor']).replace(',', ' '))
        diferenca_valor = abs(valor_planilha - valor_novo)

        if (row[3] == dados['nome'] and
            row[4] == dados['referencia'] and
            diferenca_valor < 0.01):

            # Verificações de NF e data de vencimento (código existente)
            nf_corresponde = True
            dt_venc_to_corresponde = True

            if 'nf' in dados and dados['nf'] and row[5]:
                nf_corresponde = (str(dados['nf']).strip().upper() == str(row[5]).strip().u

            if 'dt_venc_to' in dados and row[9]:
                if isinstance(row[9], datetime):
                    data_planilha = row[9].strftime('%d/%m/%Y')
                else:
                    data_planilha = str(row[9])

                dt_venc_to_corresponde = (str(dados['dt_venc_to']).strip() == data_planilha.s

            if nf_corresponde and dt_venc_to_corresponde:
                logger.warning(f"Duplicata detectada: {dados['nome']} - {dados['referencia']}
                return True
    except Exception as e:

```

```

        logger.error(f"Erro ao verificar duplicidade: {str(e)}")
        continue

    return False

```

## 1.3 Modificar o método enviar\_dados

No método `enviar_dados`, adicione as colunas STATUS e ID\_LANCAMENTO quando salvar na planilha.

Localize a seção onde os dados são salvos e adicione:

```

python

# Após salvar os dados normais (aproximadamente na linha onde salva observação)
# Adicionar as novas colunas
sheet.cell(row=proxima_linha, column=14, value='ATIVO') # STATUS
sheet.cell(row=proxima_linha, column=15, value=proxima_linha - 1) # ID_LANCAMENTO

```

## 2. Modificações no relatorio\_despesas\_aprimorado.py

### 2.1 Substituir métodos na classe RelatorioHandler

Substitua os seguintes métodos pelos modificados:

- `carregar_dados_excel` → `carregar_dados_excel_modificado`
- `calcular_acumulado_dados` → `calcular_acumulado_dados_modificado`
- `processar_dados` → `processar_dados_modificado`
- `processar_lancamentos_futuros` → `processar_lancamentos_futuros_modificado`
- `criar_tabela_despesas` → `criar_tabela_despesas_modificado`

### 2.2 Modificar a classe RelatorioUI

No método `setup_ui`, adicione após o checkbox "Incluir lançamentos futuros":

```

python

# Checkbox para Lançamentos excluídos
self.incluir_excluidos = BooleanVar(value=False)
ttk.Checkbutton(main_frame, text="Incluir lançamentos excluídos no relatório",
                variable=self.incluir_excluidos).pack(pady=5, anchor='w')

```

Substitua o método `gerar_relatorio` pela versão `gerar_relatorio_modificado`.

## 3. Estrutura das novas colunas na planilha

As planilhas dos clientes terão as seguintes colunas adicionais:

| Coluna | Nome                | Descrição                    |
|--------|---------------------|------------------------------|
| 14     | STATUS              | 'ATIVO' ou 'EXCLUIDO'        |
| 15     | ID_LANCAMENTO       | Identificador único numérico |
| 16     | HISTORICO_ALTERACAO | Timestamps de alterações     |

## 4. Funcionalidades implementadas

### 4.1 Gerenciador de Lançamentos

- **Visualização:** Lista todos os lançamentos com filtros por data e status
- **Edição:** Permite editar qualquer campo de um lançamento
- **Exclusão lógica:** Marca lançamentos como excluídos sem removê-los
- **Restauração:** Reativa lançamentos excluídos
- **Auditoria:** Registra timestamps de alterações

### 4.2 Relatórios adaptados

- **Filtro automático:** Por padrão, exclui lançamentos marcados como excluídos
- **Opção de inclusão:** Checkbox para incluir excluídos quando necessário
- **Identificação visual:** Lançamentos excluídos aparecem destacados quando incluídos
- **Cálculos corretos:** Valores acumulados consideram apenas lançamentos ativos

### 4.3 Proteção contra duplicidade

- **Verificação inteligente:** Ignora lançamentos excluídos ao verificar duplicatas
- **Permite re-lançamento:** Lançamentos similares a excluídos podem ser adicionados

## 5. Como usar

### 5.1 Para editar um lançamento:

1. Selecione um cliente na aba "Seleção de Cliente"
2. Vá para aba "Fornecedor"
3. Clique em "Gerenciar Lançamentos"
4. Selecione o lançamento na lista
5. Clique em "Editar"
6. Faça as alterações necessárias
7. Clique em "Salvar"

## 5.2 Para excluir um lançamento:

1. Siga os passos 1-4 acima
2. Clique em "Excluir"
3. Confirme a exclusão

## 5.3 Para gerar relatório com/sem excluídos:

1. No módulo de relatórios
2. Marque/desmarque "Incluir lançamentos excluídos no relatório"
3. Gere o relatório normalmente

## 6. Vantagens da implementação

1. **Auditoria completa:** Todos os lançamentos ficam registrados com histórico
2. **Segurança:** Dados nunca são perdidos, apenas marcados como excluídos
3. **Flexibilidade:** Relatórios podem incluir ou excluir conforme necessário
4. **Rastreabilidade:** Timestamps de todas as alterações
5. **Interface intuitiva:** Gerenciamento centralizado e fácil de usar
6. **Integridade:** Sistema de duplicatas adaptado para nova estrutura

## 7. Backup recomendado

Antes de implementar, faça backup de:

- Arquivo `Sistema_Entrada_Dados.py`
- Arquivo `relatorio_despesas_aprimorado.py`
- Todas as planilhas de clientes existentes

## 8. Migração de dados existentes

Para planilhas existentes sem as novas colunas, o sistema:

- Automaticamente adiciona colunas STATUS e ID\_LANCAMENTO
- Define STATUS = 'ATIVO' para todos os registros existentes
- Atribui IDs sequenciais baseados na linha da planilha